

Day 13 — Content Modifier: Local & Global Variables

1. Standard Expressions (Recap + New Ones)

What You Need	Expression
Current timestamp (in UTC)	<code>\${date:now:yyyy-MM-dd'T'HH:mm:ss}</code> (CPI tenants run in UTC, so this gives UTC time)
Current timestamp (in IST)	<code>\${date-with-timezone:now:IST:yyyy-MM-dd'T'HH:mm:ss}</code>
Message ID (MsgID)	<code>\${property.SAP_MessageProcessingLogID}</code>
iFlow name	<code>\${camelId}</code> — note: small "c" (it is <code>camelId</code> , not <code>CamelID</code> ; expression names are case-sensitive)

Tip: the short code `IST` can be ambiguous in Java (it can also mean Irish or Israel time). The unambiguous zone ID for India is `Asia/Kolkata` — if IST ever gives an unexpected time in your tenant, switch to `${date-with-timezone:now:Asia/Kolkata:yyyy-MM-dd'T'HH:mm:ss}`.

MsgID is the **Message Processing Log ID** — the same ID you see for that message in the Monitor tab. It is useful for tracing exactly one message.

2. The Requirement — "Process the Previous Message"

Requirement: whenever we process a message, we also need the details of the **previous message**. Example: current message has Emp ID ②; we also need to know what the earlier message (Emp ID ①) was.

What Is a "Message"?

Message (or Payload) = whatever data you are sending from the Source System to the Target System.

From the CPI perspective, the incoming payload travels together with **message headers and message properties** — that's why in CPI we call the whole package a **message made of properties, headers and body**.

3. Why Exchange Property Can't Solve This

Question from class: if we create a variable in an Exchange Property, store the previous message's details in it, and call it in the current message output — does it work? **Yes or No?**

Answer: No. CPI does not store the data between runs. Variables created in the message header, exchange property or body **exist only while that one message is being processed**. When the flow reaches the **End Message** they are **deleted** — header values and body are sent to the Target System, and everything else is simply discarded.

4. The Solution — Local Variable & Global Variable

To bring the previous message's details into the current message, we use these two **Source Types in Content Modifier**:

- **Local Variable**
- **Global Variable**

Why Don't They Get Deleted at End Message?

Answer: Properties, headers and payload are created **inside** the iFlow run, so they disappear when the run ends. **Local and Global variables are created/stored outside the iFlow** — in the tenant's persistent storage — so they survive after the message is finished and are available to the next run.

5. The Worked Scenario

We want the message body to show **current** and **old** message details together:

```
Hi,
Welcome to SAP CPI class.
Please find the below details.

Current message details
iFlow name   :
MsgID        :
Current Timestamp (in UTC) :
Current Timestamp (in IST) :
EmpID        :

Old message details
(same keys — but the values must be the PREVIOUS message's values)
```

Three things we need to know:

1. How to **write** the data to a local variable
2. How to **read** the data from a local variable
3. Where the local variable is **stored**

6. ① Writing — Write Variables (Persistence Palette)

Persistence tab (Palette) → Write Variables. This is the step used to write data to a variable.

What to Store

- **iFlow name** — same for all messages, so we **ignore it** (it's common)
- **MsgID** — a **new ID is generated for every message**, so this is the important one: get the (previous) MsgID and store it in a variable
- Previous message date in UTC, in IST, and the previous Emp ID — also stored

Example Entry in Write Variables

Field	Value
Name (key)	oldmsgid
Type	Property (Expression)
Value	\${property.SAP_MessageProcessingLogID}

For the timestamps and Emp ID we create **multiple variables** the same way, using the **standard expressions** from Section 1 — e.g. `previousMsgDate_inUTC`, `previousMsgDate_inIST`, `previousEmpID`, `previousMsgID`.

Important: Naming Rules for Variables

Not Allowed	Use Instead
Brackets ()	Plain letters, digits, underscore
Spaces, e.g. <code>in IST</code> or <code>(in IST)</code>	<code>inIST</code>

7. ② Reading — Content Modifier

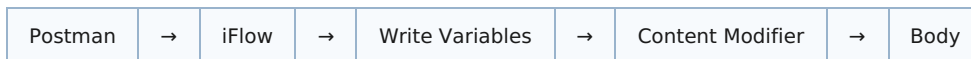
Go to the Content Modifier (created earlier) → **Exchange Property** tab → create all the fields → **Source Type = Local Variable**, and give the variable's name as the value.

Default Value → N/A (verified): when you choose Local/Global Variable as the source type, there is a **Default Value** field. SAP's documentation says the default is used **if the variable is not found at runtime**. This matters for the very **first** run (nothing has been written yet) — your trainer sets `N/A` so the flow doesn't fail and the body simply shows "N/A" as the previous message.

Timeline Example

Time	Message	"Previous message" it should show
9 AM	msg 1	N/A (first run)
10 AM	msg 2	msg 1's details
11 AM	msg 3	msg 2's details

The Flow, Start to End



Whenever we hit the sender request from Postman, the required details are stored by **Write Variables**, and we call those variables in an **Exchange Property** and then in the **Message Body**.

Ordering caution (please test this in your tenant): the order of steps decides *which* value you read. The variable always holds the **last value written**. If Write Variables runs **before** the Content Modifier that reads it, the reader may get the **current** message's value instead of the **previous** one. The safe pattern for "show the previous message" is:

1. **Content Modifier** — *read* the old variable values into Exchange Properties (with Default = N/A)
2. **Write Variables** — *store* the current message's values for the next run
3. **Content Modifier** — build the message body from the properties (old) and current values

Your notes also record the class question "*what happens if we use Write Variable before the Content Modifier?*" with the answer "*No — flow will be complex; first check whether the data is available or not.*" That is consistent with the caution above: always make sure the read happens before the write, and check the Trace (Day 8) to confirm what each step sees.

8. ③ Where Are Variables Stored?

Monitor → Manage Stores → Variables — all variables written by your iFlows are listed here.

Verified: variables are stored only for a limited time — a variable **expires after 400 days**, and the retention period keeps extending every time the iFlow containing it is processed successfully.

9. Local vs. Global Variable

	Local Variable	Global Variable
Scope	Within that iFlow only	Outside that iFlow — tenant level, shared by all iFlows on the tenant
How it's set	Write Variables with Global Scope unchecked	Write Variables with the Global Scope option checked
Visible to other iFlows?	No	Yes
Read using Content Modifier source type	Local Variable	Global Variable

How to tell them apart: looking at the **visibility in the Monitor tab** (Manage Stores → Variables), we can clearly say whether a variable is local or global. For a local variable, the **name of its iFlow** is shown in the Integration Flow column.

10. Real-Time Scenarios — Where Do We Use These?

Scenario ① — Basic Authentication

The server needs a **username and password** (Basic Auth). Credentials like these are normally kept in **Security Material (User Credentials)** — no variable needed.

Scenario ② — Token-Based Auth (OAuth2) → Global Variable

The server uses **token-based authentication (OAuth2)**; example token value: `567`. **Several iFlows need this same token** — at that point the token acts as a **Global Variable**: one iFlow fetches and stores it, and all other

iFlows read it instead of each requesting its own.

Practical caution: OAuth tokens **expire** (often within an hour), while a variable lives up to 400 days. So a stored token must be **refreshed** by the iFlow that fetches it (e.g., on a timer, or when a call returns 401). This ties directly to the token-handling issues you debugged earlier.

Scenario ③ — Local Variable: Full Load vs. Delta Load

When we want to send data, there are two options:

Full Load	Delta Load
Read the complete data . E.g., a table has 10 records — read all 10, without any issue.	Use a filter : read only the particular records updated after a certain time .

Delta load is exactly where the "previous message timestamp" variable becomes useful: store the timestamp of the last successful run in a variable, and next time fetch only what changed since then.

Correction — the delta condition is reversed in your notes. You wrote: "*condition: if current timestamp < previous message timestamp.*" The current time is always **later** than the previous run's time, so that condition would never match. The correct delta condition is:

record's last-changed timestamp > previous run timestamp (i.e., previous timestamp < record's timestamp ≤ current time)

This picks only records that changed *since* the last run.

11. Next Topics & Homework

Topic	Notes
XSLT (Mapping)	XML → XSLT → XML — coming next in class
Self-study (asked to prepare):	
① Process Direct Adapter	Calling one iFlow from another on the same tenant
② AMQP (Queues)	Message queue connectivity
③ JDBC Adapter	Connecting to databases
④ IDoc Adapter	SAP-native IDoc exchange

Want help preparing any of these four? Tell me which one and I'll explain it slowly with a real-time example.